

Репозиторий: https://github.com/AlexBerry2018/codecheck_project

CodeCheck: отчёт по проекту

Краткий итог

CodeCheck — платформа проверки заданий по программированию на трёх микросервисах Python: преподаватель создаёт курс и задание с тестами, студент отправляет решение, система проверяет его в изолированной песочнице и показывает вердикт и рейтинг. Сервисы общаются через Kafka, данные лежат в PostgreSQL и MongoDB, кэш — в Valkey, вход — через Kong.

Код и документация готовы, автоматические проверки проходят (101 тест, `guff` без замечаний). Первый запуск на реальном Docker нашёл ошибку в `grader-service` (ключ сообщения Kafka был числом); она исправлена и покрыта тестом. Повторный прогон `scripts/e2e.py` на полном стенде после исправления был выполнен успешно.

Соответствие заданию

Все восемь пунктов high-level score закрыты; пункты 4–8 реализованы в коде, пункты 1–3 — в документе проекта.

Архитектура

Все внешние запросы идут через Kong; у `grader-service` нет маршрутов наружу. Сервисы вызывают друг друга по HTTP только за карточкой задания (`submissions-service`) и тестами (`grader-service`) у `courses-service`, всё остальное — события в Kafka: решение уходит в `submission.created`, вердикт возвращается в `submission.graded`, смена тестов — в `assignment.updated`.

Данные и кэш

Каждый сервис владеет своими данными: базы PostgreSQL отдельные, у каждой своя роль, поэтому ссылок между базами нет (идентификаторы передаются числами).

Хранилище	Что хранит	Владелец	Срок жизни
PostgreSQL <code>courses_db</code>	пользователи, курсы, задания, тесты, outbox	<code>courses-service</code>	бессрочно
PostgreSQL <code>submissions_db</code>	решения, вердикты по версиям тестов, outbox	<code>submissions-service</code>	бессрочно
MongoDB <code>sources</code>	исходный код решений	<code>submissions-service</code>	TTL 1 год
MongoDB <code>run_logs</code>	логи запусков по тестам	<code>grader-service</code>	TTL 90 дней
Valkey	кэш, лимиты, рейтинг, ключи идемпотентности	все три	по TTL, вытеснение <code>volatile-lru</code>

PostgreSQL держит то, что нельзя потерять и что нужно связывать транзакциями; MongoDB — крупные документы с произвольной формой и автоматическим удалением.

В Valkey есть семь семейств ключей: `assignment:` (карточка задания, 60 с), `tests:`, `status:` (статус решения), `lb:` (рейтинг — sorted set, перестраивается из PostgreSQL при потере), `idem:`, `rl:` и `regrade:`. Если `courses-service` недоступен, закешированная карточка задания позволяет принимать решения.

Кafka и надёжность доставки

Сервисы обмениваются событиями через один брокер Kafka (KRaft). Доставка — «как минимум один раз», поэтому все обработчики идиempотентны.

Топик	Партиций	Кто пишет	Кто читает
<code>assignment.updated</code>	3	<code>courses-service</code>	<code>grader-service</code>
<code>submission.created</code>	6	<code>submissions-service</code>	<code>grader-service</code>
<code>submission.created.retry</code>	6	<code>grader-service</code>	<code>grader-service</code>
<code>submission.created.dlq</code>	1	<code>grader-service</code>	—
<code>submission.graded</code>	6	<code>grader-service</code>	<code>submissions-service</code>

Как событие не теряется и не применяется дважды:

- **Transactional outbox.** Событие пишется в таблицу в той же транзакции, что и решение; отдельный поток-релей публикует его в Kafka (`SELECT ... FOR UPDATE SKIP LOCKED`, поэтому несколько копий не отправляют одно и то же дважды).
- **Идиempотентные потребители.** Вердикт хранится по ключу (решение, версия тестов), повторная доставка упирается в первичный ключ.
- **Повтор и DLQ.** Временный сбой ведёт в `submission.created.retry` с задержкой 5 с × номер попытки (consumer ждёт отметки времени в сообщении, прежде чем взять его в работу); после трёх попыток сообщение уходит в DLQ, а студент получает вердикт `system_error`, а не вечное ожидание.
- **Idempotency-Key.** Повтор отправки с тем же ключом возвращает то же решение и не тратит квоту.
- **Перепроверка.** При смене тестов версия растёт, преподаватель запускает `regrade`, и старые решения проверяются заново.

Сравнение с RabbitMQ и NATS приведено в документе проекта: Kafka выбрана из-за журнала событий с хранением 7 дней, групп коньюмеров и гарантии порядка внутри партиии.

Роутинг и безопасность

Единая точка входа — Kong (порт 8000, без базы, конфигурация в `kong/kong.yml`). Он перенаправляет запросы на нужный сервис, балансирует между копиями и проверяет токен до того, как запрос дойдёт до кода.

Маршрут	Поведение	Лимит
/api/auth	без JWT, вход и регистрация	5 в минуту на IP
/api/courses, /api/assignments	JWT обязателен	100 в минуту на IP
POST /api/submissions	JWT, отправка решения	10 в минуту на токен; точный лимит 10 в минуту на пользователя считает сам сервис
GET /api/submissions, /api/leaderboard	JWT, чтение	100 в минуту на IP

Счётчики лимитов Kong хранит в Valkey; если Valkey недоступен, лимитер пропускает запросы (fail-open). Kong OSS не умеет считать лимит по полю внутри JWT, поэтому точный лимит вынесен в сервис.

Меры безопасности:

- **JWT** (HS256, iss=codecheck, доступ 15 минут, обновление 7 дней) проверяется и в Kong, и в каждом сервисе; алгоритм зафиксирован, exp, iss, sub обязательны.
- **Внутренние вызовы** /internal/* защищены токеном X-Internal-Token (сравнение в постоянное время) и наружу через Kong не опубликованы.
- **Песочница** запускает каждый тест в отдельном Docker-контейнере: без сети, файловая система только для чтения, cap-drop ALL, пользователь nobody, лимиты памяти, процессов и времени, вывод ограничен.
- **Базы и брокер** не видны из сети: порты PostgreSQL, MongoDB, Valkey, Kafka и сервисов проброшены на хост только на 127.0.0.1 (для проверки с вашей машины), наружу смотрят только Kong (8000) и демо-страница (8080).

Инструменты разработки

В проект добавлены контроль качества и удобство проверки на локальной машине.

Инструмент	Для чего	Как запустить
Kafka UI	топики, сообщения, группы консьюмеров и lag; режим «только чтение», порт только на 127.0.0.1	http://localhost:8081 после docker compose up
pre-commit	ruff, hadolint, проверка YAML и JSON, поиск приватных ключей перед коммитом	pre-commit install
GitHub Actions	ruff, hadolint, docker compose config, pytest по трём сервисам, сборка образов; полный прогон	.github/ workflows/ci.yml

Инструмент	Для чего	Как запустить
	стенда — вручную	
Postman	16 запросов по всем сценариям	postman/ codecheck.postman_ collection.json
scripts/e2e.py	сквозной проход UC-01... UC-06, идемпотентности, перепроверки, лимита запросов	python scripts/e2e.py -- rate-limit

Порты для проверки с хоста: Kong 8000, демо-страница 8080, Kafka UI 8081, courses 8001, submissions 8002 (Swagger на /docs), grader 8003, PostgreSQL 5432, MongoDB 27017, Valkey 6379, Kafka 29092. Если порт занят, его можно заменить в .env.

Проверка

Автоматически проверены логика сервисов, конфигурация compose и демо-страница;

Проверка	Результат
grader-service, pytest	59 прошли
courses-service, pytest	27 прошли, 1 пропущен
submissions-service, pytest	15 прошли, 2 пропущены
ruff	без замечаний
docker compose config	конфигурация валидна
Демо-страница	прогнана в headless Chromium против заглушки API
Запуск стенда на машине автора	песочница готова, консьюмеры запущены, задание получено из courses-service; до исправления вердикт не публиковался из-за ошибки ключа, после — повторный прогон scripts/e2e.py прошёл успешно

Ошибки первого запуска и исправления

Первый запуск на машине автора выявил две ошибки в коде и одну в сборке образов; все три разобраны ниже.

Симптом	Причина	Исправление
Сборка падает с таймаутом auth.docker.io	строка # syntax=docker/dockerfile:1, добавленная в Dockerfile, заставляет Docker скачивать отдельный образ с Docker Hub	строка удалена из трёх Dockerfile; базовые образы по-прежнему тянутся с Docker Hub, его должно быть видно с машины

Симптом	Причина	Исправление
Решение остаётся <code>queued</code> , е2е: «не проверено за 120 с»	<code>grader</code> падал на <code>key.encode: submission_id</code> приходит числом, а ключ сообщения Kafka должен быть строкой; сообщение циклилось в повторах	ключ приводится к строке в <code>worker.py</code> и в трёх <code>kafka_io.py</code> ; тесты теперь берут числовые <code>id</code> , как в реальных событиях
<code>grader</code> не стартует без доступа к Docker Hub	при неудачном <code>docker pull</code> он отказывался работать, даже если образ песочницы уже лежал на хосте	берётся локальная копия с предупреждением; добавлено 2 теста (на машине автора образ был скачан, причиной была ошибка ключа)

Ограничения и доработки

Это осознанные упрощения стенда.

Ограничение	Почему так и что делать в продакшене
Сокет Docker у <code>grader</code> (равно <code>root</code> на хосте)	удобно для демо; нужны <code>rootless Docker</code> , <code>gVisor</code> или отдельный хост для запусков
Лимит на пользователя считает сервис, а не Kong	Kong OSS не считает по полю <code>JWT</code> ; Kong ограничивает по <code>IP</code> и по токену
Преподаватель видит вердикты чужих курсов	исходный код в ответ не входит; нужна проверка владельца курса через <code>courses-service</code>
Нет статуса <code>running</code>	статус <code>queued</code> до вердикта
Рейтинг только по заданию	рейтинга по курсу нет
Пиковая память теста не измеряется	превышение лимита определяется по <code>OOM</code>
MongoDB без пароля, общий экземпляр	роли по сервисам сделаны только для PostgreSQL
Схемы БД создаются через <code>create_all</code>	миграций нет, следующий шаг — Alembic
Kafka — один брокер без репликации	это стенд; в продакшене нужны минимум три брокера
Постоянный баг в обработчике блокирует партицию	если обработчик падает с исключением (не временным сбоем проверки), <code>consumer</code> повторяет то же сообщение в процессе и не идёт дальше по партиции; в логе это было видно при ошибке ключа
Только Prometheus-метрики	сам Prometheus и дашборды в <code>compose</code> не входят
Нет OpenAPI у Flask-сервиса	описание есть в README и в коллекции

Ограничение	Почему так и что делать в продакшене
	Postman; Swagger есть у submissions-service

Как запустить

Нужен Docker с доступом к Docker Hub (или заранее скачанные образы, включая `python:3.12-alpine` для песочницы).

1. В корне проекта: `docker compose up --build`.
2. В логе `grader-service` дождаться строки `sandbox ready`.
3. `python scripts/e2e.py --rate-limit`: проходит все сценарии UC-01...UC-06, идемпотентность и лимит запросов. Код выхода 0 означает, что всё прошло.
4. Демо-страница `http://localhost:8080` (вход, курс, задание, отправка решения, вердикт, рейтинг), затем Kafka UI `http://localhost:8081`, где видно, как событие проходит через `submission.created` и `submission.graded`.
5. Для показа отказоустойчивости: остановить `grader-service`, отправить решение (оно принимается и ждёт в Kafka), запустить `grader` снова — вердикт появится.

Если решения зависают в `queued`, смотрите `docker compose logs grader-service --tail 50`: причина будет написана в строке `sandbox is not ready` или в трейсбеке обработчика.

System Design

CodeCheck принимает решение за доли секунды, а проверяет его асинхронно, поэтому вся архитектура строится вокруг очереди в Kafka и изолированного исполнителя. Ниже — путь посылки, расчёт нагрузки, компромиссы и поведение при отказах; общее устройство сервисов — в разделе «Архитектура» выше.

Требования и допущения

Функциональные: преподаватель создаёт курс и задание с тестами; студент отправляет решение на Python и получает вердикт по каждому тесту; есть история попыток, рейтинг по заданию и перепроверка после смены тестов.

Нефункциональные, как они зашиты в код и конфигурацию:

Требование	Значение в проекте
Ответ на отправку	HTTP 202 сразу, статус <code>queued</code> , вердикт приходит позже
Размер решения	до 64 КБ, больше — 413
Частота отправок	не больше 10 в минуту на пользователя
Доставка событий	как минимум один раз, обработчики идемпотентны
Изоляция чужого кода	отдельный контейнер на каждый тест, без сети, с лимитами памяти, процессов и времени

Требование	Значение в проекте
Хранение	события Kafka 7 дней, исходники 1 год, логи запусков 90 дней
Статус решения	кэшируется в Valkey: 3 с пока ждёт очереди, 10 мин после вердикта

Числа нагрузки (число студентов, пик перед дедлайном) в задании не заданы; ниже они взяты как допущения для расчёта. Допущения согласованы с ТЗ проекта: пик около 1 решения в секунду (300 за 5 минут), для нагрузочного прогона берётся запас до 5 в секунду.

Оценка нагрузки

Пик перед дедлайном — около 1 решения в секунду (запас для прогона — до 5), и даже на запасе производительность упирается в grader, а не в API. Всё ниже — расчёт по допущениям, на боевом трафике ничего не измерялось; единственное измеренное число — 315 мс на запуск пустого контейнера (`baseline_ms` из лога grader на машине автора).

Допущения

Допущение	Значение
Студентов в системе	2 000
Отправляют в пик	300 (столько отправляют за 5 минут перед дедлайном)
Частота отправок в пике	300 решений за 300 с, то есть 1 в секунду (лимит 10 в минуту на пользователя — верхняя граница); для нагрузочного прогона запас до 5 в секунду
Тестов в задании	4 в среднем
Время одного запуска теста	около 0,4 с (315 мс на контейнер плюс код решения)
Размер решения	5 КБ (лимит 64 КБ)
Решений в сутки	5 000

Расчёт

Величина	Как считается	Результат
Пик отправок	$300 \div 300 \text{ с}$	≈ 1 решение в секунду (запас: 5)
Запусков песочницы	1×4 теста	4 в секунду (при запасе 20)
Контейнеров одновременно	$4 \times 0,4 \text{ с}$ (закон Литтла)	≈ 2 (при запасе: ≈ 8)
Память песочницы, верхняя оценка	$2 \times 64 \text{ МБ}$ (лимит, не фактическое потребление)	$\approx 128 \text{ МБ}$ (при запасе $\approx 0,5 \text{ ГБ}$)
Одна копия grader	одно решение за раз, 4 теста по 2 параллельно = $2 \times 0,4 \text{ с}$	$\approx 0,8 \text{ с}$ на решение, $\approx 1,2$ в секунду

Величина	Как считается	Результат
Копий grader на 1 в секунду	$1 \div 1,2$	$\approx 0,8$: хватает одной копии (загрузка около 80 %), для запаса две; на 5 в секунду — около 5
Поток в Kafka	$1 \times 5 \text{ КБ}$	$\approx 5 \text{ КБ/с}$ на <code>submission.created</code>
Хранение в Kafka, 7 дней	$5\,000 \times 5 \text{ КБ} \times 7$	$\approx 0,2 \text{ ГБ}$
Исходники в MongoDB, 1 год	$5\,000 \times 5 \text{ КБ} \times 365$	$\approx 9 \text{ ГБ}$
Опрос статуса, раз в 2 с, верхняя оценка	$300 \text{ ждущих} \div 2 \text{ с}$	≈ 150 запросов в секунду, большинство из Valkey

Узкие места и масштабирование

По оценке, PostgreSQL, Valkey и HTTP-слой выдержат пик. Grader при 1 решении в секунду загружен одной копией примерно на 80 %, поэтому запускают две; потолок шести партиций — около 7 решений в секунду, то есть запас $\times 7$.

№	Узкое место	Почему	Что делать
1	Партиции <code>submission.created</code>	партиций 6, каждая копия grader берёт одно решение за раз, поэтому больше 6 копий не ускоряют: $6 \times 1,2 \approx 7$ решений в секунду	на пике 1 решение в секунду запаса хватает; партиции (до 12–16) и копии grader поднимать, если пик вырастет выше 7 в секунду; уменьшить число партиций нельзя, поэтому решать заранее
2	Хост с Docker	контейнеры запускаются на одном демоне; при лимите 1 CPU на контейнер на запасе в 5 решений в секунду одновременно до 8 запусков, то есть до 8 ядер по лимиту; на пике 1 в секунду около 2	отдельные воркеры для запусков со своим демоном Docker, а не сокет боевого хоста
3	Лимит Kong на чтение по IP	100 запросов в минуту на адрес; опрос статуса раз в 2 с — 30 запросов в минуту на	лимит по IP — известное ограничение стенда (ТЗ описывает именно

№	Узкое место	Почему	Что делать
		студента, и четверо студентов за одним NAT (кампусная сеть) упрутся в 429	его): считать лимит по токену или заменить опрос на SSE или WebSocket
4	Один брокер Kafka и одна PostgreSQL	одна точка отказа на каждый	три брокера и репликация 3, PostgreSQL с репликой и автопереключение
5	Запись исходника в MongoDB при отправке	приём решения зависит от доступности MongoDB: без неё ответ 503	реплика-сет MongoDB или хранение исходников в объектном хранилище

Масштабирование горизонтальное: `docker compose up --scale grader-service=N` (до числа партиций), а также `--scale courses-service` и `--scale submissions-service`, после того как у масштабируемого сервиса убран проброс портов на хост. Kong балансирует между копиями через upstream с health checks, а внешних обращений к grader нет — он читает только Kafka.

Ключевые решения и компромиссы

Каждое решение купило надёжность или простоту ценой задержки или сложности; здесь эта цена написана рядом с выбором.

Решение	Альтернатива	Зачем	Цена
Приём решения с 202 и асинхронная проверка	синхронный ответ с вердиктом	проверка долгая и непредсказуемая, API не должен зависеть от неё	клиент опрашивает статус
Kafka	RabbitMQ, NATS	журнал событий с хранением 7 дней, группы потребителей, порядок внутри партиции, перечитывание при перепроверке	тяжелее в эксплуатации, число партиций задаёт потолок параллелизма
Transactional outbox	публикация в Kafka прямо из обработчика запроса	решение и событие записываются в одной транзакции, событие не теряется, если Kafka недоступен	дополнительная таблица и поток-релей, задержка до полусекунды при простое релея

Решение	Альтернатива	Зачем	Цена
Доставка «как минимум один раз» и идемпотентные обработчики	exactly-once	exactly-once на стыке Kafka и двух баз не достижимо просто	каждый обработчик обязан выдержать повтор (ключ решение + версия тестов)
События вместо HTTP между submissions и grader	прямой вызов grader	grader может быть медленным или недоступным, очередь сглаживает пики	вердикт приходит с задержкой, нужен опрос статуса
Контейнер на каждый тест	пул прогретых контейнеров	чистая изоляция: ничего не остаётся между запусками	около 315 мс на старт контейнера, они вычитаются из времени теста
Лимит на пользователя считает сервис	лимит по полю JWT в Kong	Kong OSS так не умеет	логика лимитов в двух местах
PostgreSQL и MongoDB	всё в одной базе	реляционные данные с транзакциями отдельно от крупных документов с автоудалением по TTL	два хранилища на эксплуатацию
Рейтинг в sorted set Valkey	считать в SQL на каждый запрос	чтение топа быстро, данные без TTL не вытесняются (volatile-lru)	при потере Valkey рейтинг перестраивается из PostgreSQL, первый запрос медленнее

Отказы

Что происходит, когда отдельный компонент недоступен.

Компонент	Что происходит	Чем защищено
Kafka недоступна	Приём решений продолжается: посылка и событие пишутся в PostgreSQL одной транзакцией, событие ждёт в outbox. После возвращения брокера relay-поток отправляет накопленное.	Transactional outbox, FOR UPDATE SKIP LOCKED
grader-service упал	Сообщения остаются в submission.created	Offset-ы в Kafka, at-least-

Компонент	Что происходит	Чем защищено
	, растёт lag. После рестарта потребитель продолжает с сохранённого offset.	once
Сбой при проверке (песочница, таймаут инфраструктуры)	Сообщение уходит в <code>submission.created.retry</code> с линейной задержкой. После 3 попыток событие попадает в DLQ, а посылка получает вердикт <code>system_error</code> .	Retry-топик, DLQ
Повторная доставка события	Результат не пересчитывается и не дублируется.	Идемпотентный потребитель по ключу (<code>submission_id</code> , <code>tests_version</code>)
Valkey недоступен	Кэш пропускается, данные читаются из PostgreSQL. Rate limiter в Kong работает в режиме fail-open: запросы не блокируются. Лидерборд пересобирается из PostgreSQL.	Cache-aside, источник истины в БД
Инстанс сервиса не отвечает	Kong исключает его из балансировки по health check.	Upstream health checks
Контейнер песочницы завис или упёрся в лимит	Контейнер убивается по таймауту. Лимиты CPU, памяти и числа процессов не дают ему повлиять на хост и соседей.	- - network none, read-only, - - cap-drop ALL, лимиты

Известные слабые места

- Одиночный брокер Kafka и один экземпляр каждой БД: отказ любого из них останавливает соответствующую часть системы.
- Повторы внутри consumer блокируют партицию. Если ошибка детерминированная (например, баг в тестах), одно сообщение задерживает остальные в этой партиции до исчерпания попыток.
- Mongo без авторизации, а `docker.sock` у grader-service даёт ему права на хостовый Docker. Для учебного стенда это допустимо, для продакшена нет.

Что поменять для продакшена

- **Kafka:** 3 брокера, `replication factor 3`, `min.insync.replicas=2`, больше партиций под рост нагрузки.
- **PostgreSQL:** реплика для чтения и резервные копии; миграции через Alembic вместо `create_all`.

- **MongoDB:** replica set и авторизация.
- **Песочница:** rootless Docker, gVisor или отдельные хосты-раннеры вместо `docker.sock` внутри `grader-service`.
- **Доступ:** проверка владельца курса для преподавателей (сейчас они видят вердикты чужих курсов), секреты из менеджера секретов, а не из переменных окружения.
- **Лимиты:** лимит по пользователю (по полю `JWT`) в Kong (плагин `Enterprise` или собственный) либо `SSE` вместо опроса статуса.
- **Consumer:** отличать детерминированную ошибку от временной и отправлять первую сразу в `DLQ`, не блокируя партицию повторами.
- **Наблюдаемость:** Prometheus и Grafana поверх существующих `/metrics`, алерты на `lag`, размер `outbox` и объем `DLQ`.